

Android Application Development – Fundamentals

Day 01 - Session 02

21st September 2026

C-DAC, Hyderabad

01 Android Application Building Blocks

02 Intents & IPC

03 Android Application Structure

04 Modern Android development with Kotlin & Jetpack Compose

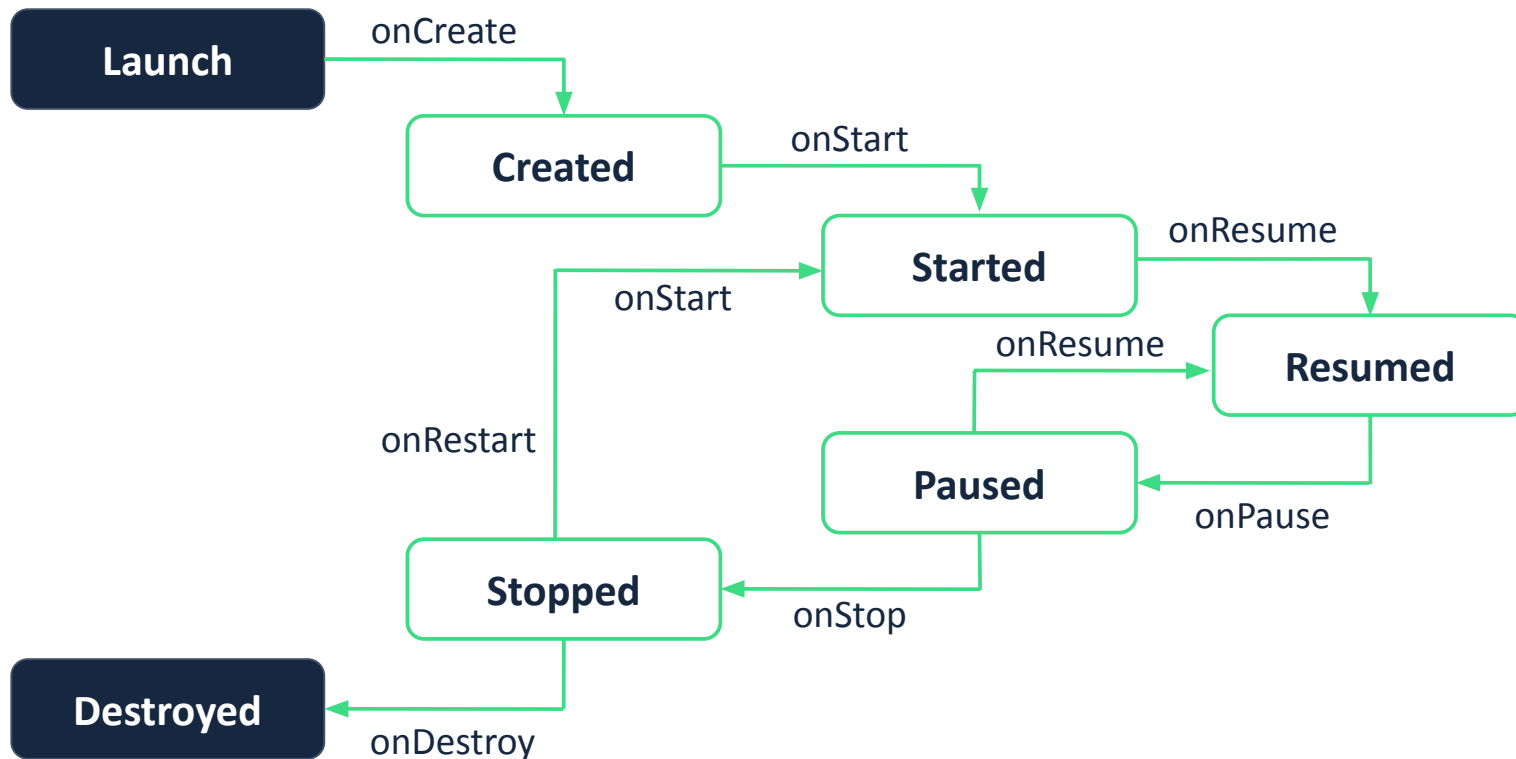
Android Application Building Blocks

Activities



- An Activity represents a single screen which provides the interface through which users interact with the application.
- Activities can communicate with each other using Intents.
- Extend the **Activity** class to create activities.

Activity Lifecycle



Intents & IPC

Intents are messages used to communicate between Android components

They allow components within an app or between different apps to interact

Intents can start Activities, Services, and Broadcast Receivers

Intent filters define which intents a component can respond to

Implicit Intent

- Specifies an action, not a target component.
- Android finds a suitable component to handle it.
- Often used for inter-app/system actions.
- Examples: share, open webpage, make a call, view location.

```
Intent intent = new Intent(Intent.ACTION_DIAL);  
intent.setData(Uri.parse("tel:9876543210"));  
startActivity(intent);
```

Explicit Intent

- Specifies the target component directly.
- Mainly used within the same app.
- Provides direct, predictable navigation.
- Used to start a specific Activity, Service, or Receiver.

```
Intent intent = new Intent(packageContext: MainActivity.this, SecondActivity.class);  
startActivity(intent);
```


Inter Process Communication

- IPC allows different processes to communicate with each other.
- To perform IPC, your application must bind to a service using `bindService()`.
- A method called by an Activity or another application component is executed in a different process, and the result is returned to the calling component.

- Service is an Android application component used to perform background or long-running operations without a user interface (UI). It allows an application to perform certain tasks even when the user is interacting with another application.
- Services can operate in two main modes:
 - Started Service: Started using `startService()` and runs until it is stopped.
 - Bound Service: Connected to another component, such as an Activity, for communication.



Playing music while browsing



File download in progress



Location tracking

Services

Types of Android Services

Foreground Service

- Noticeable to the user and continues running even when the app is not in use.
- Must display a notification that the service is running.

Background Service

- Not directly noticeable to the user
- Android places restrictions on background services, especially when the app is not in foreground.

Bound Service

- Provides a client-server interface that allows components to communicate and interact with service.
- Runs while application components are bound to it and is destroyed when all components unbind from it.

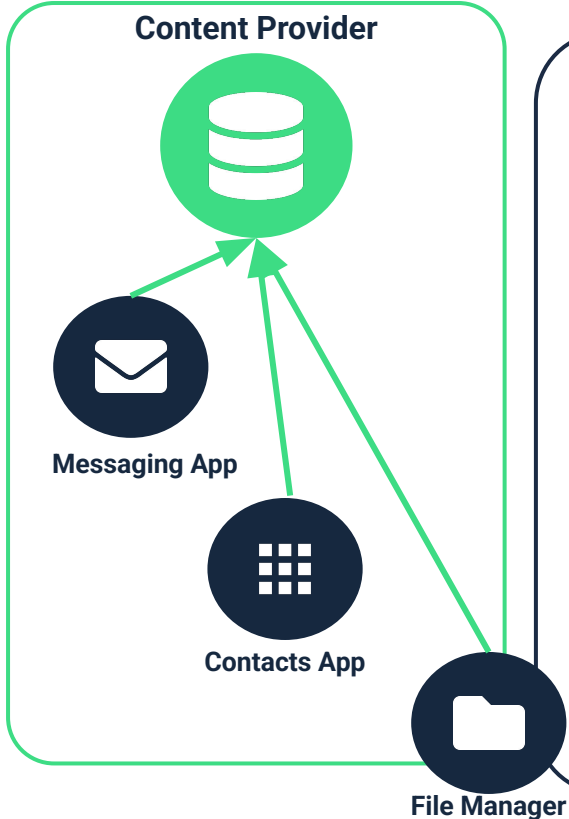
Broadcast Receivers

- Broadcast Receivers can respond to system-wide broadcasts, such as device booting, network connectivity changes, battery low notifications, etc.
- Broadcast Receivers can be registered statically in the AndroidManifest.xml file or dynamically at runtime.
- They are often used in combination with other components like Services or Content Providers to create more robust and interactive applications

**System
Broadcast Events**



Content Providers



- Android Content Providers are components that manage access to structured data and facilitate sharing data between different apps.
- Content Providers can be accessed by other apps through ContentResolver, which is the primary interface to communicate with Content Providers.
- Content Providers expose structured data through standard operations such as query, insert, update, and delete, while controlling how that data can be accessed.

Android Application Structure

The screenshot displays the Android Studio IDE interface. On the left, the 'Project' view shows the directory structure of the 'DemoApp' project. The 'app' directory is expanded, revealing sub-directories like 'manifests' (containing 'AndroidManifest.xml'), 'java' (containing the 'com.demo.demoapp' package with 'MainActivity'), and 'res'. The 'MainActivity.java' file is selected and open in the editor. The code in the editor shows the package declaration, imports, and the implementation of the 'onCreate' method, which sets up the content view and window insets. The bottom status bar indicates the current file path and settings.

```
activity_main.xml MainActivity.java
1 package com.demo.demoapp;
2
3 > import ...
10
11 public class MainActivity extends AppCompatActivity {
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         EdgeToEdge.enable( $this$enableEdgeToEdge: this);
17         setContentView(R.layout.activity_main);
18         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsets
19             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
20             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
21             return insets;
22         });
23     }
24 }
```

Android Application Structure

Manifest File

```
AndroidManifest.xml ×
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3    xmlns:tools="http://schemas.android.com/tools">
4    <uses-permission android:name="android.permission.INTERNET" />
5    <application
6      android:allowBackup="true"
7      android:dataExtractionRules="@xml/data_extraction_rules"
8      android:fullBackupContent="@xml/backup_rules"
9      android:icon="@mipmap/ic_launcher"
10     android:label="DeviceAttTest"
11     android:roundIcon="@mipmap/ic_launcher_round"
12     android:supportsRtl="true"
13     android:theme="@style/Theme.DeviceAttTest">
14     <activity
15       android:name=".MainActivity"
16       android:exported="true"
17       android:label="@string/app_name"
18       android:theme="@style/Theme.DeviceAttTest">
19       <intent-filter>
20         <action android:name="android.intent.action.MAIN" />
21
22         <category android:name="android.intent.category.LAUNCHER" />
23       </intent-filter>
24     </activity>
```

Modern Android Development



Live Demonstration: Creating a Basic Android Application

Modern Android development with Kotlin & Jetpack Compose



Modern Android Development

Why Kotlin ?

Kotlin is the primary recommended language for modern Android development



Officially supported by Google for Android development



Supports object-oriented and functional programming



Null-safety helps prevent common crashes



Concise and readable syntax



Interoperable with existing Java code

What is Jetpack ?

Android Jetpack is a collection of libraries and tools that helps developers build Android applications more easily and reliably

ViewModel

Manages UI-related data

Navigation

Move between screens

Room

Work with local databases

WorkManager

Schedule background work

Lifecycle

Handle component lifecycle

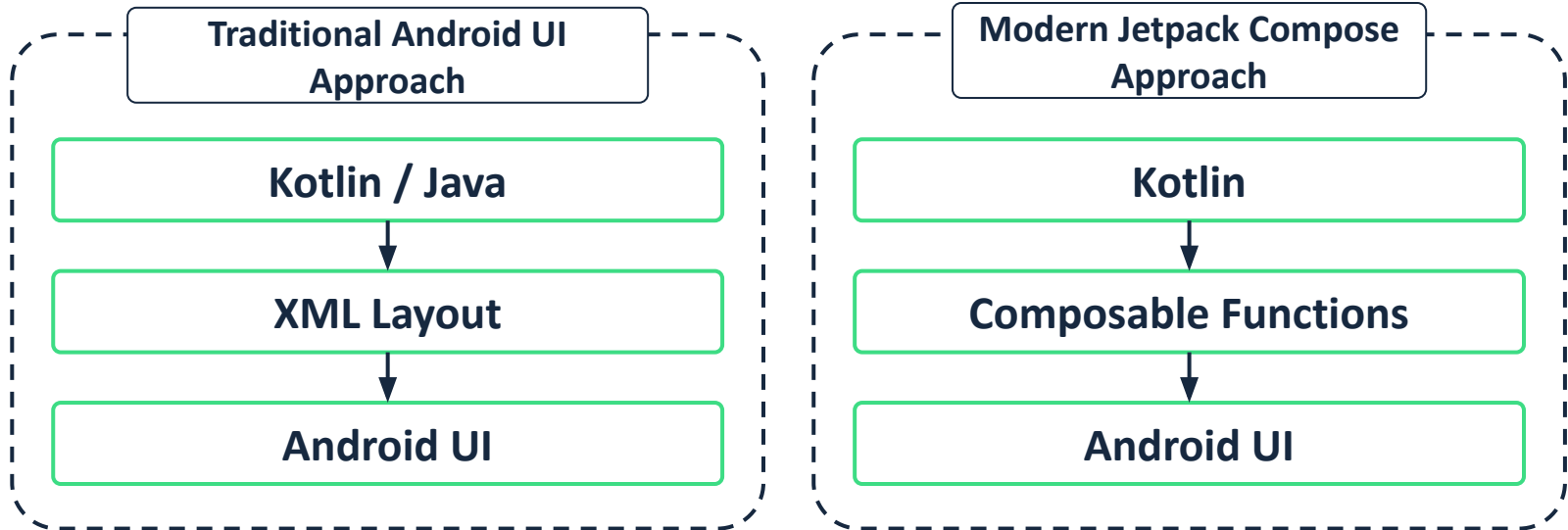
DataStore

Store application data

Modern Android Development

What is Jetpack Compose?

Jetpack Compose is Android's modern toolkit for building native user interfaces. Instead of XML layouts, developers describe the UI using Kotlin code



Modern Android Development

Key Differences: XML Views vs Jetpack Compose

Traditional XML

- XML Layouts
- `findViewById()`
- View-based UI
- UI updated manually in many cases
- Seperate layout & Code

Jetpack Compose

- Kotlin Code
- Direct composable references
- Declarative UI
- UI reacts to state
- UI & logic expressed together

THANK YOU!!